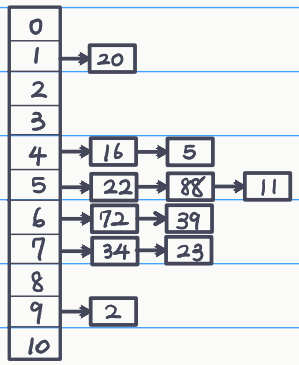
1. Draw the 11-entry hash table that results from using the hash function, $h(i) = (2i + 5) \bmod 11$, to hash the keys 34, 22, 2, 88, 23, 72, 11, 39, 20, 16, and 5, assuming collisions are handled by the following approaches respectively.

(a) *chaining*.

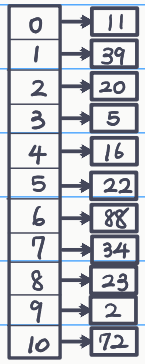
Input	34	22	2	88	23	72	11	39	20	16	5
Hash Code	7	5	9	5	7	6	5	6	1	4	4

Hash Table :



(b) *linear probing*.

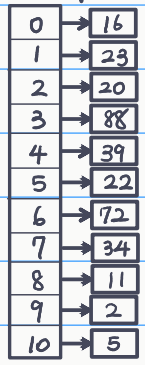
Hash Table :



(c) *double hashing* using the secondary hash function $h'(k) = 7 - (k \mod 7)$. Note that double hashing uses $(h(k) + j \times h'(k)) \mod N$, for $j = 1, 2, \ldots, N-1$, when collisions occurs.

Input	34	22	2	88	23	72	11	39	20	16	5
Hash Code	7	5	9	5→3	7→1	6	5→8	6→9→1→4	1→2	4→9→2→7→1→6→0	4→6→⋯→10

Hash Table :



2. Let D be an ordered dictionary with n entries implemented by means of an AVL tree. Show how to implement the following operation on D in time $O(\log n + s)$, where s is the size of the entries returned:

findAllInRange(k_1, k_2): Return all the entries in D with key k such that $k_1 \leq k \leq k_2$.

HINT: First find the ends of the range and then find a way to efficiently enumerate the entries in between.

```
FUNCTION Find All In Range (k1, k2, v):
    if v is None:
        return []
    buffer = []
    if v.key > k1:
        buffer.extend (Find All In Range (k1, k2, v.left))
    if v.key ≤ k2 and v.key ≥ k1:
        buffer.append (v)
    if v.key < k2:
        buffer.extend (Find All In Range (k1, k2, v.right))
    return buffer
```

3. Prove that the number of (single or double) rotations done in removing a key from an AVL tree can not exceed half the height of the tree.

1. 設 T 是一高度為 h 的 AVL 樹
2. 節點 N 只會在其子樹高度變化導致樹高相差 > 2 時需要旋轉
3. 設 N 的高度為 k，且需要旋轉，其子樹高度必為 k-1 與 k-2，且操作使 k-2 的樹高改為 k-3
4. 也就是說 k-2 樹高的子樹也可能受影響而需要旋轉
5. 設高度為 h 的節點需要 R(h) 次旋轉，可寫出 $R(h) = 1 + R(h-2)$
6. $R(h) = 1 + R(h-2) < \frac{h}{2}$ #

4. Recall the red-black tree structure we discuss in class. It manages n entries, of which each is a pair of *key* and *value*, (key, value) and supports, in $O(\log n)$ time, the operations `search(x)`, `insert(x)` and `delete(x)`, where x is a key for some entry.
Describe how to augment a red-black tree to support, in $O(\log n)$ time, the operations `increase(x, y, q)` and `decrease(x, y, q)`. That is to have additional fields (or attributes) used in the structure. In `increase(x, y, q)`, the value ($w.value$) of every entry w in the tree with key, $x \leq w.key \leq y$, is increased by q (the `decrease` operation does the corresponding decrease). One cannot explicitly update the entries as this may take $O(n)$ time. Describe how **all** operations can be executed in $O(\log n)$ time.

1. 對於每個節點新增一個 buffer 成員變數

2. Function Increase (x, y, q , root):

```
node = root
while node is not None:
    Update ( curr )
    if node.key < x :
        node = node.right
    else if node.key > y :
        node = node.left
    else :
        node.value += q
        break
if not node : Return
curr = node.left
while curr is not None :
    Update ( curr )
    if curr.key >= x :
        curr.value += q
        if curr.right is not None :
            curr.right.buffer += q
            curr.right.value  += q
        curr = curr.left
    else :
        curr = curr.right
```

```
curr = node.right
while curr is not None :
    Update ( curr )
    if curr.key <= y :
        curr.value += q
        if curr.left is not None :
            curr.left.buffer += q
            curr.left.value  += q
        curr = curr.right
    else :
        curr = curr.left
```

```
Function Update (node) :
    node.left.value  += node.buffer
    node.right.value += node.buffer
    node.left.buffer  += node.buffer
    node.right.buffer += node.buffer
    node.buffer = 0
```

<下一頁尚有作答>

3. 使用這套做法可以在尋訪到目標子樹時，不用遍歷每個子節點進行更新，而是對要更新的節點的 buffer 加上特定的值，並在下次尋訪時更新並傳遞更新子節點。

在 Search、Deletion、Insertion 中也需要加入 Update 操作，以確保獲得正確的值。